

Département d'informatique 420-126-LI - Programmation Orientée Objet 1	Note de cours - Semaine 1
Écrites par Robert Aubé	Modifiées par Julien Brunet

Table des matières

Glossaire des termes utilisés dans le cours	2
Introduction	4
Qu'est-ce qu'un programme informatique?.....	4
Comment passer d'un programme source à un programme exécutable par la machine?	5
Environnements de développement	5
Bases de programmation en Java	6
Une instruction Java.....	6
Les lignes blanches.....	6
Les commentaires	6
Commentaires de fin de ligne ou de ligne	6
Commentaires sur plusieurs lignes	7
Instruction d'affichage à la console	7
Opérations simples sur des variables numériques (+, -, *, /)	8
Les variables.....	9
Conventions et règles de nommage des variables	10
Modifier le contenu d'une variable ou « Affectation ».....	11
Trace d'exécution	11
Introduction au type de variable String	12
Création d'une variable String	12
Concaténation d'une chaîne de caractères	12
Afficher le contenu d'une variable	13

Glossaire des termes utilisés dans le cours

Terme	Définition
Accesseur	Méthode dont le rôle est de donner à l'appelant une valeur contenue dans les attributs de l'appelé
Adresse mémoire	Nombre correspondant à l'emplacement de l'élément dans le tableau de la mémoire de l'application.
Affectation	Instruction qui donne une nouvelle valeur à une variable. L'affectation écrase la valeur déjà présente.
Algorithme	Suite finie et non ambiguë d'opérations ou d'instructions permettant de résoudre un problème
Appel de méthode	Instruction qui lance l'exécution d'une autre méthode (et possiblement à un autre objet qu'on nomme alors l'appelant). Un appel peut transmettre des valeurs (sous forme de paramètres) à la méthode appelée. La méthode appelée peut ou non retourner une valeur à son appelant.
Attribut	Variable appartenant à un objet. L'attribut existe en mémoire aussi longtemps que l'objet qui le possède.
« bytecode »	Code intermédiaire (en java) entre le code source et le code binaire propre à la plateforme
Classe	En programmation orientée objet (POO), la déclaration d'une classe regroupe des attributs et méthodes (comportements) communs à un ensemble d'objets (instances). <u>Cela permet de définir un nouveau type de données</u> pour créer une série d'objets (instances) de ce type. La classe peut être comparée à une recette de cuisine (recette pour faire un biscuit) avec laquelle <u>on peut cuisiner plusieurs fois des éléments similaires</u> . (plusieurs biscuits similaires, des objets, des instances).
Code binaire	Code composé de 0 et 1 prêt à être exécuté par un processeur donné.
Code source	Code compréhensible par le programmeur respectant les standards d'un langage donné. Ce code est très précieux, il ne faut jamais le perdre (le supprimer).
Compilateur	Outil servant à transformer le code source en code binaire. Le compilateur transforme tout le code source en un programme exécutable d'un seul coup. En Java c'est du « bytecode ».
Constructeur	Méthode servant à donner une valeur initiale correcte à l'ensemble des attributs d'un objet en création. En Java il a exactement le même nom que la classe.
Débogueur	Outil servant à exécuter le code ligne par ligne et à afficher le contenu des variables au fur et à mesure. Le débogueur est une sorte d'interpréteur avec lequel on contrôle l'exécution des instructions.
Déclaration	Instruction qui informe le compilateur de l'existence d'une nouvelle variable. En Java la déclaration comprend obligatoirement un nom et un type.
Erreur d'exécution	Le programme compile, mais le compilateur ne perçoit aucune erreur. Cependant lors de l'exécution, le programme se termine de manière anormale et brutale. Souvent causé par une opération illégale (ex. Division par zéro).
Erreur de compilation	Erreur dans la syntaxe du code source qui empêche le compilateur de produire le « bytecode » ou le code binaire.
Erreur de logique	Ce sont les erreurs les plus difficiles à détecter et à corriger. Le programme compile et s'exécute sans problème apparent, sauf que le résultat obtenu n'est pas celui escompté! (Exemple : un calcul qui ne donnerait pas le bon résultat). Pour ce type d'erreur, le débogueur est un outil intéressant.

Initialisation	Instruction qui affecte une valeur à une variable pour la première fois.
Instance	Une instance est une réalisation en mémoire d'une classe, un objet. Toutes les instances d'une même classe partagent une définition commune, mais chacune possède ses propres valeurs, son propre état (Synonyme d'objet).
Instruction	Élément minimal du code source. En java les instructions sont séparées par des « ; ».
Interpréteur	Outil servant à exécuter du code source. L'interpréteur traduit le code source ligne par ligne. Le code est exécuté au fur et à mesure que l'interpréteur traduit le code source. Le débogueur Java est une sorte d'interpréteur. NOTE : En Java l'interpréteur interprète le « bytecode ».
Machine virtuelle	Programme qui sert à interpréter du « bytecode » (.class) pour le rendre exécutable par un processeur précis.
Méthode	Comportement associé à un objet
Mutateur	Méthode dont le rôle est de modifier une valeur des attributs de l'objet utilisé
Objet	Un objet est une réalisation en mémoire d'une classe. (Synonyme d'instance)
Opérateur	En programmation, les opérateurs sont utilisés pour effectuer des opérations sur des "opérandes" (variables, constantes, objets...). Il existe plusieurs types d'opérateurs parmi lesquels : • Opérateurs arithmétiques : Elles sont utilisées pour effectuer des opérations mathématiques telles que l'addition, la soustraction, la multiplication et la division. • Opérateurs de comparaison : Ils sont utilisés pour comparer deux valeurs et retourner un résultat vrai ou faux. • Opérateurs logiques : Ils sont utilisés pour effectuer des opérations logiques, telles que ET, OU, et NON. • Opérateurs d'affectation : Ils sont utilisés pour attribuer des valeurs à des variables ou d'autres types de données. • Opérateurs Bitwise : Elles sont utilisées pour effectuer des opérations « bitwise » sur des valeurs binaires.
Paramètre	Variable reçue par une méthode et déclarée entre parenthèses à côté du nom de la méthode (Synonyme d'argument).
Plateforme	Combinaison d'éléments (système d'exploitation, processeur, mémoire, machine virtuelle...) qui détermine le cadre d'exécution d'un programme.
Procédure	Groupe d'instructions qui réalisent un objectif commun. La procédure débute lorsqu'elle est appelée et se termine en retournant à son point d'appel (Synonyme de méthode ou fonction). Dans le monde de la POO on parle de méthodes .
Référence	Adresse mémoire emmagasinée dans une variable servant à accéder à un objet. La référence n'est pas l'objet, elle permet simplement d'accéder à l'objet (aux éléments internes de l'objet, les attributs, les données). Une même variable peut changer d'objet en cours d'exécution, donc changer de référence. On dit aussi qu'une variable pointe sur un objet, par référence.
Retour	Valeur retournée par une méthode. L'instruction « return » en java est généralement placée à la fin de la méthode, indiquant la valeur retournée.
Signature	Chaîne de caractères formée par la visibilité, le type de retour, le nom et les paramètres d'une méthode.
Type	Lors de la déclaration d'une variable ou une constante, cela définit la nature de valeurs que peut prendre une variable.
Variable	Espace mémoire réservé (lors d'une déclaration de variable) pour contenir une information qui peut être changée lors de l'exécution du programme. Une variable a toujours un nom, un type (qui définit la nature de ce qu'elle peut contenir) et une valeur.

Introduction

Qu'est-ce qu'un programme informatique?

- C'est un ensemble d'instructions destinées à être exécutées par un ordinateur. Ces instructions ont une syntaxe assez stricte pour que l'ordinateur soit capable de les comprendre.
- On distingue 2 sortes de programmes :
 - Un **programme source** est un code écrit par un **informaticien** dans un langage de programmation. Il peut être compilé vers une forme binaire. Exemple d'un **programme source Java** :

```
package semaine01.exemples;

public class Exemple1 {

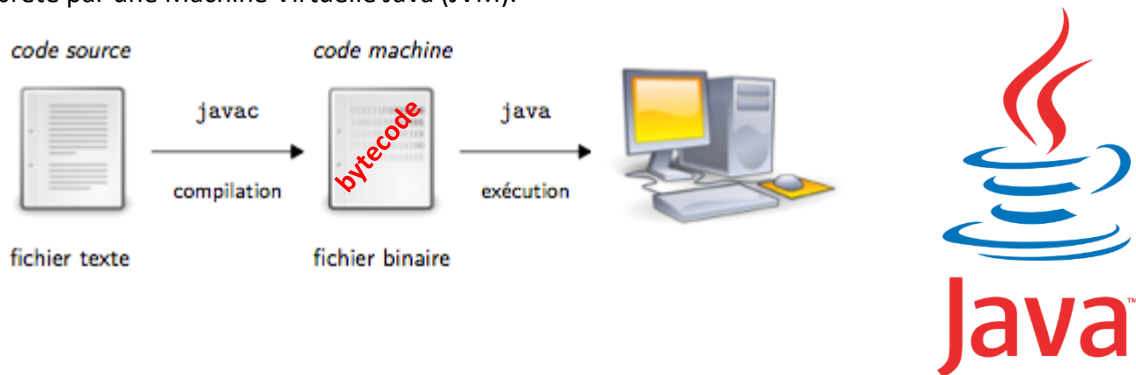
    public static void main(String[] args) {
        int annee = 2026;
        System.out.println("Voici un programme java de " + annee);
    }
}
```

- Un **programme binaire** décrit les instructions à exécuter par un **ordinateur** sous forme numérique. Ces instructions définissent un langage machine.

```
00000000 fc 31 c0 8e c0 8e d8 8e d0 bc 00 7c 89 e6 bf 00 |.1.....|...|
00000010 06 b9 00 01 f3 a5 89 fd b1 08 f3 ab fe 45 f2 e9 |.....E..|
00000020 00 8a f6 46 bb 20 75 08 84 d2 78 07 80 4e bb 40 |...F. u...x..N.@|
00000030 8a 56 ba 88 56 00 e8 fc 00 52 bb c2 07 31 d2 88 |.V..V....R...l..|
00000040 6f fc 0f a3 56 bb 73 19 8a 07 bf 87 07 b1 03 f2 |o...V.s.....|
00000050 ae 74 0e b1 0b f2 ae 83 c7 09 8a 0d 01 cf e8 c5 |.t.....|
00000060 00 42 80 c3 10 73 d8 58 2c 7f 3a 06 75 04 72 05 |.B...s.X,...u.r.|
00000070 48 74 0d 30 c0 04 b0 88 46 b8 bf b2 07 e8 a6 00 |Ht.0...F.....|
00000080 be 7b 07 e8 b2 00 8a 56 b9 4e e8 8e 00 eb 05 b0 |.{.....V.N.....|
00000090 07 e8 b0 00 30 e4 cd 1a 89 d7 03 7e bc b4 01 cd |....0.....~...|
000000a0 16 75 0d 30 e4 cd 1a 39 fa 72 f2 8a 46 b9 eb 16 |.u.0...9.r..F...|
000000b0 30 e4 cd 16 88 e0 3c 1c 74 f1 2c 3b 3c 04 76 06 |0.....<.t...;<.v.|
000000c0 2c c7 3c 04 77 c9 98 0f a3 46 0c 73 c2 88 46 b9 |,<.w....F.s..F..|
000000d0 be 00 08 8a 14 89 f3 3c 04 9c 74 0a c0 e0 04 05 |.....<.t.....|
000000e0 be 07 93 c6 07 80 53 f6 46 bb 40 75 08 bb 00 06 |.....S.F.@u....|
000000f0 b4 03 e8 59 00 5e 9d 75 06 8a 56 b8 80 ea 30 bb |...Y.^..u..V...0.|
00000100 00 7c b4 02 e8 47 00 72 86 81 bf fe 01 55 aa 0f |.|...G.r.....U..|
00000110 85 7c ff be 85 07 e8 19 00 ff e3 b0 46 e8 24 00 |.|.....F.$..|
00000120 b0 31 00 d0 eb 17 0f ab 56 0c be 78 07 e8 eb ff |.l.....V..x....|
00000130 89 fe e8 03 00 be 85 07 ac a8 80 75 05 e8 04 00 |.....u.....|
00000140 eb f6 24 7f 53 bb 07 00 b4 0e cd 10 5b c3 8a 74 |..$.S.....[.t|
00000150 01 8b 4c 02 b0 01 56 89 e7 f6 46 bb 80 74 13 66 |..L...V...F..t.f|
00000160 6a 00 66 ff 74 08 06 53 6a 01 6a 10 89 e6 48 80 |j.f.t..Sj.j...H..|
00000170 cc 40 cd 13 89 fc 5e c3 20 20 a0 0a 44 65 66 61 |.@....^...Defa|
00000180 75 6c 74 3a a0 0d 8a 00 05 0f 01 06 07 0b 0c 0e |ult:.....|
00000190 83 a5 a6 a9 0d 0c 0b 0a 09 08 0a 0e 11 10 01 3f |.....?|
000001a0 bf 44 4f d3 4c 69 6e 75 f8 46 72 65 65 42 53 c4 |.D0.Linu.FreeBS.|
000001b0 66 bb 44 72 69 76 65 20 00 00 80 8f b6 00 00 00 |f.Drive .....|
000001c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
000001f0 00 00 00 00 00 00 00 00 00 00 00 00 00 55 aa |.....U..|
00000200
```

Comment passer d'un programme source à un programme exécutable par la machine?

- Un **compilateur** permet de passer d'un **programme source** à un **code machine** exécutable par la machine.
- En java, le compilateur s'appelle « **javac** ». Ce dernier produit un code intermédiaire « **bytecode** » qui sera interprété par une Machine Virtuelle Java (JVM).



- Opérations pour exécuter un programme :
 1. Le programmeur écrit dans un fichier texte qui contient des instructions dans un certain langage, par exemple Java (**.java**)
 2. Il le compile avec **javac.exe**. Ce qui produit un code machine Java, le « **bytecode** » et le met dans un **fichier binaire (.class)**.
 3. Ce **code machine** Java ou « **bytecode** » est interprété et exécuté par le programme **java.exe** que l'on appelle la **machine virtuelle** (JVM). L'exécution, si le code source n'a pas d'erreur, devrait correspondre au résultat attendu par le programmeur.
- Les **environnements de développement** (IDE) offrent au programmeur un outil puissant pour le supporter dans l'écriture des programmes informatiques.
- **IDE = Integrated Development Environment** (Environnement de développement intégré)

Environnements de développement

- Un **environnement de développement** est un programme qui regroupe un ensemble d'outils pour augmenter la productivité des programmeurs qui développent des logiciels.
- Minimalelement, il comporte :
 - un **éditeur de texte** destiné à la programmation, écrire du code.
 - un **bouton ou option de menu** qui permet de démarrer le compilateur puis qui exécute le programme
 - un **débogueur**, qui permet d'exécuter ligne par ligne le programme en cours d'exécution
- Il y a plusieurs environnements de programmation pour développer en java :
 - **IntelliJ IDEA** (<https://www.jetbrains.com/idea/download/#section=windows>)
 - Eclipse (<https://eclipse.org/downloads/>)
 - NetBeans (<https://netbeans.org/downloads/8.0.2/>)
- Dans le cadre de ce cours, nous utiliserons **IntelliJ IDEA**.

Bases de programmation en Java

Une instruction Java

- Une instruction est une opération simple que doit exécuter le programme.
- En java les instructions sont séparées par des « ; ».
- Les blocs de code en Java sont encadrés par des accolades { }.
- En général on ne met qu'une instruction par ligne de code. Même si le langage permet d'en mettre autant qu'on le souhaite, c'est une mauvaise pratique de le faire.
- Certaines lignes du code source ne sont pas des instructions : les **lignes blanches** et les **commentaires**.

Les lignes blanches

- Une ligne blanche n'est pas une instruction.
- Ces lignes sont ignorées lors de la compilation.
- Le Java permet de mettre autant de ligne vide que l'on veut.
- Elles permettent de séparer des blocs de code, d'aérer le code et d'augmenter sa lisibilité.

Les commentaires

- Un commentaire n'est pas une instruction.
- Un commentaire permet d'insérer du texte qui ne sera pas compilé.
- Il donne la possibilité au programmeur d'ajouter du texte explicatif au code source.
- Nous verrons plus loin que les commentaires permettront au programmeur de documenter son code.
- Il donne la possibilité au programmeur de **neutraliser** une ligne de code (la mettre en commentaire).
- En java, il y a 2 types de commentaires :
 - Les **commentaires de fin de ligne ou de ligne** (le //)
 - Les **commentaires sur plusieurs lignes** (bloc) (les /*et */)

Commentaires de fin de ligne ou de ligne

- Un **commentaire de fin de ligne ou de ligne** commence par un double « slash » (//) et se termine au retour à la ligne.
- **Exemple**

```
// Un commentaire pour donner l'exemple
int nbArticle = 10; // 10 articles

// Ici, on veut simplement ignorer temporairement une instruction.
// Celle-ci ne sera pas exécutée.

// System.out.print ("Au revoir!");
```

Commentaires sur plusieurs lignes

- Un **commentaire sur plusieurs lignes** est encadré par « slash + étoile » et « étoile + slash ».
- Exemple :

```
/*
Ceci est un commentaire
sur plusieurs lignes.
*/

/*
Code de débogage désactivé :

int a = 10;
System.out.println("Valeur de a" + a);
*/
```

Instruction d'affichage à la console

- On peut utiliser les méthodes **print** et **println** de l'objet **System.out** pour afficher le contenu d'une chaîne et/ou le résultat d'un calcul mathématique.
La différence entre le **System.out.println** et le **System.out.print** est que le premier ajoute un saut de ligne après l'affichage de l'information.

Exemple

```
System.out.println ("Voici un programme Java écrit en 2026.");
System.out.println ("Au mois de septembre.");
System.out.println ("Le Titanic a coulé en 1912.");
System.out.print ("Ça fait ");
System.out.print (2026 - 1912);
System.out.println (" ans.");
```

À l'exécution ce code affichera :

```
Voici un programme Java écrit en 2026.
Au mois de septembre
Le Titanic a coulé en 1912.
Ça fait 114 ans.
```

Opérations simples sur des variables numériques (+, -, *, /)

- Soit l'expression qui représente l'addition de 2 nombres entiers :

`1 + 1`

- Cette opération est une addition : « **la valeur de l'expression est** » le résultat de l'addition des **2 valeurs** « **1** » et l'expression vaut **2**.
- De la même manière, on peut utiliser toutes les opérations arithmétiques simples : « **+** **-** ***** **/** ».

Exemple

```
4 + 1 //L'expression vaut 5
6 * 3 //L'expression vaut 18
9 / 3 //L'expression vaut 3
```

- Il faut comprendre ici que les instructions précédentes sont inutiles parce qu'elle n'utilise pas le résultat. Une façon d'utiliser serait d'en faire l'affichage.

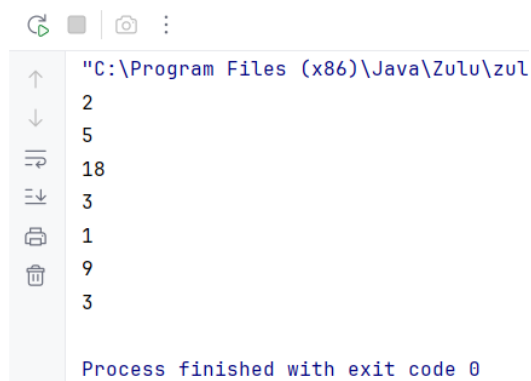
Exemple

```
System.out.println (1 + 1); //Affiche 2
System.out.println (4 + 1); //Affiche 5
System.out.println (6 * 3); //Affiche 18
System.out.println (9 / 3); //Affiche 3
```

- On peut également utiliser les parenthèses « **()** » pour forcer l'ordre des opérations (**bonne pratique**).

```
System.out.println (9 / 3 - 2); //Affiche 1
System.out.println (9 / (3 - 2)); //Affiche 9
```

Sortie :



```
"C:\Program Files (x86)\Java\Zulu\zul
2
5
18
3
1
9
3

Process finished with exit code 0
```


Les variables

- Les variables sont des symboles qui associent un nom (un identifiant) à une valeur. Sa valeur est d'un type précis de donnée (la nature de son contenu).
- On peut voir une variable comme **une boîte** dans laquelle on peut placer **une donnée**.

Exemple

```
int variable = 0;  
variable = 17;
```



- Cette boîte, on va lui donner **un nom** afin de l'identifier pour **accéder** et **modifier** son contenu dans le code.
- Dans l'exemple, on place d'abord, dans cette boîte, **la valeur 0** lors de sa déclaration et ensuite **la valeur 17**. Comme cette valeur est **un entier**, on indique que le format (la nature) de la boîte est fait pour contenir un entier. Ainsi, on **déclare** que cette variable est un **int**.
- Un nom de variable** ne peut comporter que des lettres, des chiffres et les caractères "_" et "\$". « \$ » ne devrait pas être utilisé pour des variables.
- Un nom de variable ne peut pas commencer par un chiffre et ne doit pas comporter d'espace
- Les noms de variables ne peuvent pas être les noms réservés du langage (abstract, assert, boolean, break, byte, case, catch, char, class, const, continue, default, do, double, else, enum, extends, false, final, finally, float, for, goto, if, implements, import, instanceof, int, interface, long, native, new, null, package, private, protected, public, return, short, static, strictfp, super, switch, synchronized, this, throw, throws, transient, true, try, void, volatile, while, etc.)
- En Java, toutes les variables utilisées dans un programme doivent être déclarées avec un type.**
- Dans tous les langages, la **bonne pratique** est de déclarer les variables dans le haut des méthodes.
- Le langage Java est sensible à la casse, ce qui signifie que des lettres majuscules et minuscules ne constituent pas la même variable.

Exemple :

La variable **I**, est différente de **i**.

La variable **Nombre** est différente de **nombre**.

- Leur contenu est modifié avec le signe « = » (affectation).

Exemple :

```
a = 2;
```

- En résumé, les variables doivent commencer par une **lettre** (ou un "_") et peuvent contenir **lettre**, **chiffres** et le caractère "_" (barre de soulignement). L'espace n'est pas autorisé.

Exemples :

Nom de variable	Est valide ou est refusé?
_	Est valide
_a	Est valide
nombreDeChat	Est valide
Nombre de chat	Refusé parce qu'il y a des espaces
Jean@mail-city.com	Refusé parce qu'il y a des caractères spéciaux (@, -, .)
11a	Refusé parce qu'elle commence par un nombre
true	Refusé parce que c'est un mot réservé

Exemple

Voici un exemple simple de code source Java où chaque ligne est une instruction :

```
// Déclaration des variables
int premierNombre = 0;
int secondNombre = 0;
int resultat = 0;

// Affectation des variables
premierNombre = 5;
secondNombre = 8;

// On écrase la valeur 5 par 12 dans la variable « premierNombre »
premierNombre = 12;

// Opération sur des variables
resultat = premierNombre + secondNombre;
```

Conventions et règles de nommage des variables

- Le **nom d'une variable** est un **identificateur unique** pour une portion de code. Il faudra choisir un nom qui décrit bien son contenu (auto-documentation du code).

Exemple d'une portion de code :

Mauvais	Bon
<pre>int a = 100; int b = 50; int c; c = a / b;</pre>	<pre>int distance = 100; int vitesse = 50; int tempsDeplacement; tempsDeplacement = distance / vitesse</pre>

Par convention :

- les noms de variables commencent par une minuscule
- elles contiennent des majuscules uniquement à chaque changement de mot (on appelle cela le [camelCase](#))
- on n'utilise pas d'accent. On écrit « **eleve** » ("élève" fonctionne, mais ne doit pas être utilisé)
- Il convient de choisir des noms de variable qui soient le plus concis et explicites possible (auto-documentation du code). Un nom de variable comme **maSuperVariableQuiContientMonAge** est à éviter et on préférera **monAge** ou même simplement **age**. La longueur maximale suggérée est de 32 caractères.
- Dans la mesure du possible on évite les abréviations qui peuvent introduire un doute
- Exemples de bon nom de variable :

Explication du contenu de la variable	Bon nom de la variable
Vitesse d'une voiture	vitesseVoiture
Nombre de réussites	nbReussites
Somme des factures	sommeFacture
Nom de l'employé	nomEmploye
Vrai si c'est un succès	estUnSucces
Vrai si c'est réussi	estReussi

Modifier le contenu d'une variable ou « Affectation »

- On utilise l'opérateur « = » pour modifier ou affecter le contenu d'une variable.

```
premierNombre = 5;
```

- Avec cette instruction, on demande de placer la valeur 5 dans la variable **premierNombre**.
- À gauche il ne peut y avoir qu'une variable et à droite il peut y avoir toutes formes d'expression qui fournit une valeur du type prédéterminé lors de la déclaration de la variable, ici un entier.

Trace d'exécution

On appelle une trace, un tableau qui donne l'historique des valeurs des différentes variables durant l'exécution d'un programme. Cet outil (habituellement papier) est très intéressant pour vérifier la validité d'un **algorithme très simple**.

Algorithme : suite finie et non ambiguë d'opérations ou d'instructions permettant de résoudre un problème.

Soit le code suivant :

```
public class Application {  
  
    public static void main(String[] args) {  
        int a ;  
        int b ;  
        int c ;  
  
        a = 5;           // ligne 1  
        b = a + 4;       // ligne 2  
        c = a * b - 12;  // ligne 3, « ??? » ici avec des « ( ) » ça serait plus clair...  
        a = a + a;       // ligne 4  
        b = c - a * 2;   // ligne 5, « ??? » ici avec des « ( ) » ça serait plus clair...  
    }  
}
```

Faisons la trace des 5 lignes identifiées dans le code. Pour cela on doit d'abord faire un tableau qui aura pour colonnes chacune des variables impliquées dans la section de code et pour lignes les 5 instructions concernées.

étape	a == ? (int)	b == ? (int)	c == ? (int)
Ligne 1			
Ligne 2			
Ligne 3			
Ligne 4			
Ligne 5			

Vous remarquerez :

- On ne suppose pas de valeurs initiales pour les variables qui n'ont pas été affectées.
- Les opérations utilisent toujours les valeurs des variables de la ligne précédente.
- Une variable peut intervenir dans une opération qui la modifie (ligne 4). Dans ce cas, c'est quand même sa valeur à la ligne précédente qui est utilisée.

Introduction au type de variable **String**

Nous avons vu, à travers les exemples, que pour utiliser des chaînes de caractères en Java, il suffit simplement d'écrire la séquence de caractères entre guillemets doubles. Par exemple, pour afficher le texte « Hello World ! » à l'écran, on doit écrire l'instruction suivante :

```
System.out.println("Hello World !")
```

Création d'une variable **String**

- Pour créer une nouvelle variable chaîne de caractères, on fera comme tous les types de bases en Java (aussi appelé type primaire), c'est une particularité de ce type d'objet, qui n'est pas un type primaire (on revient souvent sur ça dans les prochains cours) :

```
//affectation comme un type primaire même si c'est un objet !!!  
String maChaine = "Hello World !";
```

Concaténation d'une chaîne de caractères

La **concaténation** permet de joindre plusieurs chaînes de caractères ensemble, tout comme placer plusieurs mots ensemble permettent de former une phrase. Ainsi, concaténer deux chaînes de caractères A et B donnera une nouvelle chaîne qui contiendra les informations des chaînes A et B. Cette opération se fait en utilisant l'opérateur de concaténation « + ».

Exemple

```
//Déclaration et initialisation, bonne pratique  
String prenom = "Julien";  
String nom = "Brunet";  
  
String prenomEtNom = prenom + " " + nom;  
  
System.out.println (prenomEtNom);  
System.out.println ("J'ai " + 49 + " ans."); //Ici on a un entier aussi ???
```

- À l'exécution de cette portion de code, voici ce qui sera affiché :

```
Julien Brunet  
J'ai 49 ans.
```

Afficher le contenu d'une variable

Exemple

```
/**
 * Affiche le nombre d'années depuis que le Titanic s'est échoué
 * /
public void afficherTitanic() {
    int maintenant = 2023;
    int anneeTitanic = maintenant - 1912;

    System.out.println ("Le Titanic a coulé en 1912.");
    System.out.println ("Si nous sommes en " + maintenant + ".");
    System.out.println ("Ça fait " + anneeTitanic + " ans qu'il a coulé.");
}
```

Si on appelle cette méthode

afficherTitanic();

Le résultat est

Le Titanic a coulé en 1912.
Si nous sommes en 2023.
Ça fait 111 ans qu'il a coulé.

Si on modifie « maintenant »

maintenant = 2027;

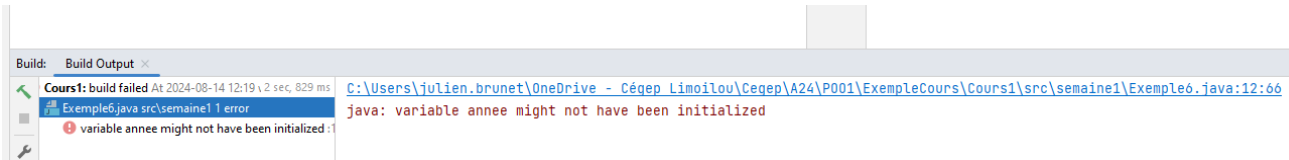
afficherTitanic();

Le Titanic a coulé en 1912.
Si nous sommes en 2027.
Ça fait 115 ans qu'il a coulé.

ATTENTION, si vous tentez d'afficher la valeur d'une variable non initialisée, cela provoquera une erreur de compilation.
En java une variable doit toujours être initialisée !!

```
String annee;  
System.out.print ("Voici un programme Java écrit en " + annee);
```

Lors de la **compilation**, l'erreur suivante sera affichée :

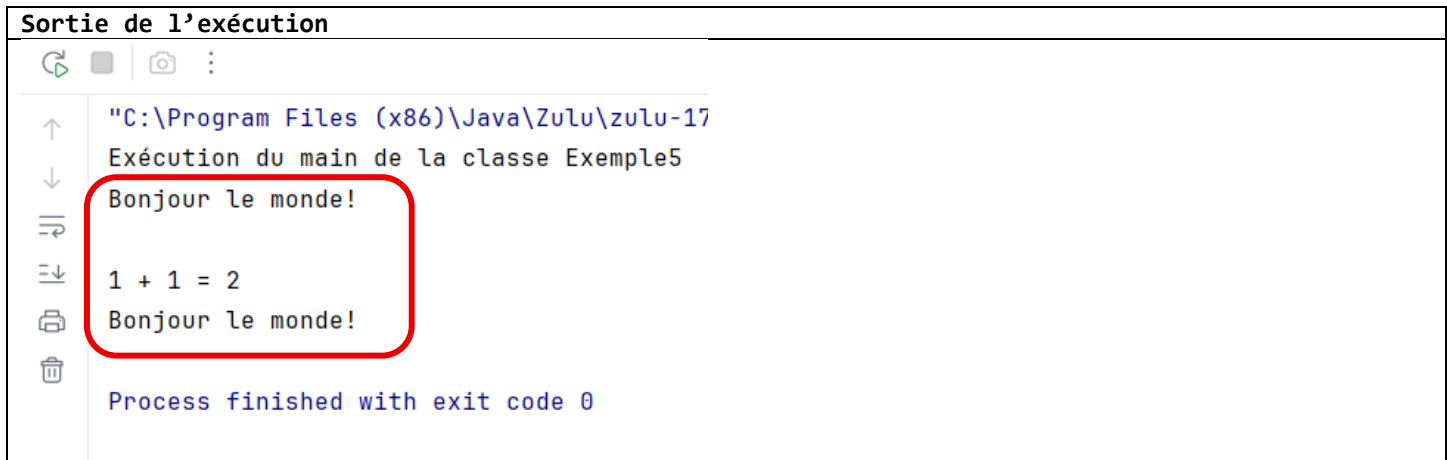


Méthodes simples (sans valeur de retour et sans paramètres)

Sans forcément comprendre l'ensemble des détails du code ci-dessous, on observe :

Exemple5.java

```
package semaine01.exemples;  
  
public class Exemple5 {  
  
    //déclaration d'une méthode appelée afficherBonjourLeMonde  
    public void afficherBonjourLeMonde() {  
        System.out.println("Bonjour le monde!");  
    }  
  
    //déclaration d'une méthode appelée afficher1Plus1  
    public void afficher1Plus1() {  
        System.out.println("");  
        System.out.print("1 + 1 = ");  
        System.out.println(1 + 1);  
    }  
  
    //déclaration de la méthode main, c'est le point d'entrée du programme (de cette  
    //classe)  
    public static void main(String[] args) {  
        System.out.println("Exécution du main de la classe Exemple5");  
  
        //instanciation (objet) de la classe Exemple5, encore un peu mystérieux...  
        Exemple5 programme = new Exemple5();  
  
        //Appel de la méthode afficherBonjourLeMonde sur l'objet appelé programme  
        //c'est ce qui va provoquer l'exécution du code de la méthode  
        programme.afficherBonjourLeMonde();  
  
        //un autre appel de méthode sur l'objet  
        programme.afficher1Plus1();  
  
        //Une même méthode peut être appelée plusieurs fois!!  
        programme.afficherBonjourLeMonde();  
    }  
}
```



On étudiera plus en détail la notion de méthode plus tard, mais pour le moment retenons :

- Une déclaration de méthode doit respecter minimalement la syntaxe suivante:

```
public void nomDeLaMethode() {  
    //Mettre ici le code de la methode  
}
```

Exemple:

```
public void bonjourLeMonde() {  
    System.out.println("Bonjour le monde!");  
}
```

- Si on ne l'appelle pas explicitement une méthode, son code, ne sera jamais exécuté
- Dans l'exemple précédent, pour pouvoir appeler une méthode, il faut fabriquer un objet de la classe dans laquelle la méthode est définie:

```
Exemple5 programme = new Exemple5();
```

- Pour l'exécuter on utilise l'objet créé « **monProgramme** » pour faire un appel de méthode par exemple:

```
programme.bonjourLeMonde();
```
- Une méthode peut être appelée autant de fois que souhaité à plusieurs endroits différents dans votre code selon vos besoins!
- Cela permet de structurer et de réutiliser!